Static Analysis for Bounds Check Elision in the Naga WGSL Compiler

Proving GPU array indices in bounds, so the clamps WebGPU forces into every shader can be dropped

Bhuvan B · ACM SIGPLAN-M Mentee | Open Source Contributor · IICT 2026 · Paper #55

29 → 0
clamps
6 GPU kernels

1 In one picture: a tree reduction loses every clamp

```
@compute @workgroup_size(256)
var<workgroup> s: array<u32, 256>;

for (var stride = 128u;
     stride > 0u;
     stride = stride >> 1u) {
  if (lid < stride) {
    s[lid] = s[lid] + s[lid + stride];
  }
  workgroupBarrier();
}
```

1. `lid` ∈ [0, 255], from the workgroup size
2. only update is `>> 1` ⇒ `stride` ∈ [1, 128] in the body
3. branch `lid < stride` ⇒ `lid` ∈ [0, 127]
4. `lid + stride` ∈ [1, 255] ⊆ [0, 255]
   ✓ **proved: no clamp needed**

SPIR-V today: 4 clamps

```
~~UMin %lid %uint_255~~
~~UMin %lid %uint_255~~
~~UMin %lid_plus_stride %uint_255~~
~~UMin %lid %uint_255~~
```

interval analysis in the validator

With elision: 0 clamps
binary byte-identical (sha256) to the build with all checks off

2 Every index pays

- WGSL promises an out-of-bounds access never leaves its resource.
- wgpu clamps every runtime index on Vulkan, Metal and D3D12. **A browser cannot switch that off.**

`s[i]`
↓ wgpu today
`s[min(i, 255)]`
↓ if i ∈ [0, 255] is proved
`s[i]`

The analysis runs in Naga's validator; the SPIR-V, MSL and HLSL writers skip the check for each proved access.

3 How a fact is proved

Every variable carries an **interval**; branches narrow it, loops match a **recurrence**.

Guard

```
var i = data[g.x];
if (i < 8u) { lut[i] }
```

i ∈ [0, 7] ✓

Bit mask

```
lut[data[g.x] & 7u]
```

∈ [0, 7] ✓

Recurrence

```
stride = stride >> 1u
```

stride ∈ [1, 128] ✓

Relation

```
if (lid >= off)
  { s[lid - off] }
```

lid − off ∈ [0, 254] ✓

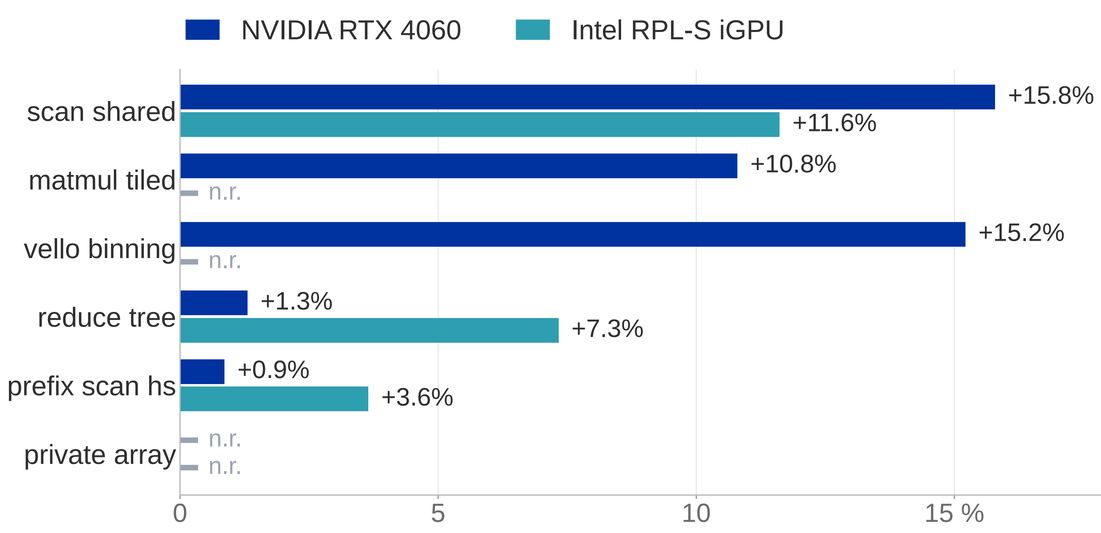
Not proved, clamp stays: runtime-sized `array<T>`, `i < arrayLength`, facts across calls, guards on `let` values.

4 Can we trust it?

- ✓ **Tests:** 95 stress shaders that must keep or lose a check; interval arithmetic checked by exhaustive enumeration
- ✓ **GPU: 8 of 8** differential fixtures byte-identical with and without elision
- ✓ **Audit:** 3 unsound shapes and an override hazard found and fixed with regression tests; **none changed a measured binary**
- ! **Open:** `discard` on Metal / D3D12; no mechanised proof

5 Results: GPU kernels and real code (vello)

Synthetic kernels: every clamp gone

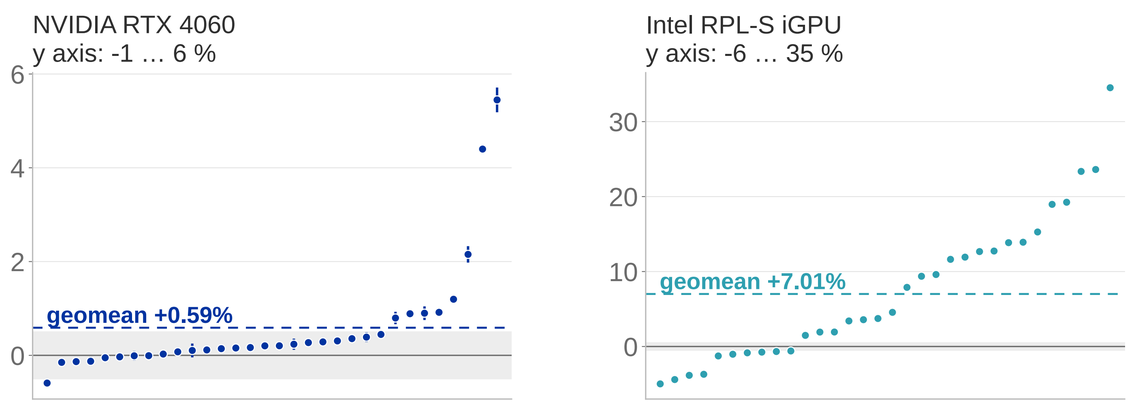


speedup from elision, geomean of valid runs · n.r. = no effect distinguishable from noise

In all 6 kernels 29 → 0 clamps; each elided binary is byte-identical (sha256) to the checks-off binary.

vello scenes, run as web content

+7.01% Intel geomean 22 faster · 10 slower
+0.59% NVIDIA geomean 22 faster · 4 slower
102/186 bounds clamps removed



One dot per scene, 95% CI. Pre-registered (98bd514); four binaries interleaved, 120 rounds; 10 runs NVIDIA, 5 Intel. 14 of the 22 NVIDIA gains are below +0.5%; 2 scenes have no usable Intel measurement.

! **Takeaway:** many checks WebGPU forces into shaders are provably redundant. A small interval analysis removes those, and every access it cannot prove keeps its check.

6 Coverage: 9 projects

729 runtime-indexed accesses
342 fixed-size arrays / vectors
118 proved in bounds

7 Related work and next steps

	This work	Tint (Chromium)		This work	Tint
any `if` guard	✓	✗	runtime-sized arrays	✗	✗
relation a ≥ b	✓	✗	shipped	✗	✓
shrinking / geometric loops	✓	✗	Tint compared by reading its source, not measured.		

Next: symbolic lengths · `let` guards · head-to-head with Tint · upstream to wgpu

364 of 729 accesses (49.9%) index runtime-sized arrays: the next target. | Hero: wgpu's Vulkan settings (index Restrict; buffers robustBufferAccess2), SPIR-V ids renamed, artifact runs-v4/20260929T125306Z-reduce_tree, fork 19ffb4e. Examples in ③ run through the analysis (poster/probes, fork f73a687). Refs: W3C WGSL spec; Tint IntegerRangeAnalysis (dawn); Bodík et al., ABCD, PLDI 2000; Würthinger et al., PPPJ 2007; LLVM IRCE; Go prove. Every number is generated from run artifacts (poster/numbers.json).

github.com/bhuvanb404